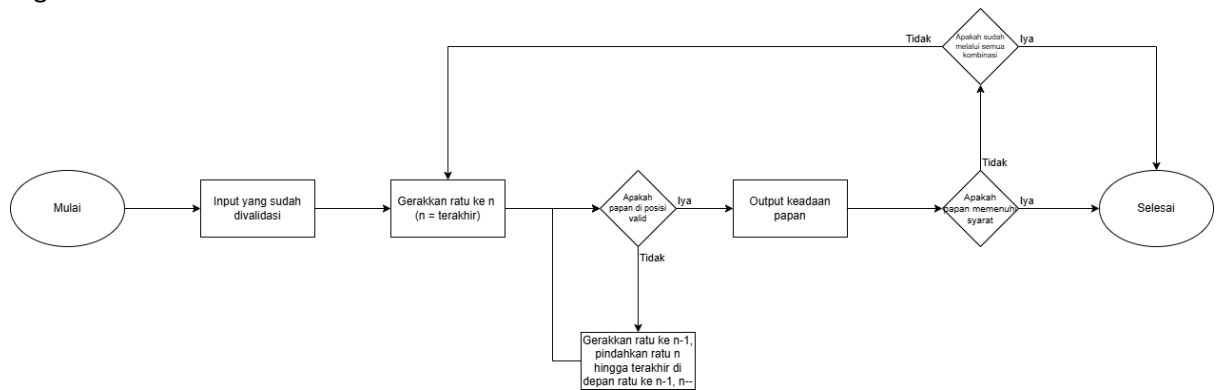


Laporan Tupil 1 Strategi Algoritma

Nathanael Shane Bennet 13524119

1. Algoritma



2. Source Code

main.c

```
#define WIN32_LEAN_AND_MEAN
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <chrono>
#include "queens.h"

int main() {
    int bsz = 0;
    queens q;
    FILE *inptr;
    char board[MAX_BOARD_SIZE * MAX_BOARD_SIZE] = {0};
    char iter_str[MAX_BOARD_SIZE] = {0};
    char in_str[100];
    char temp[200];
    int regions[26];
    int dump;

    system("cls");

    printf("Enter absolute file path, or q to quit > ");
    scanf("%s", in_str);
    if (strcmp(in_str, "q") == 0) return 0;

    int counter = 0;
    for (int i = 0; i < strlen(in_str); i++) {
        if (in_str[i] == '\\') {
            temp[counter] = '\\';
            counter++;
            temp[counter] = '\\';
        } else {
            temp[counter] = in_str[i];
        }
        counter++;
    }
    temp[counter] = '\\0';

    inptr = fopen(temp, "r");
```

```

char flag = 0;
while (flag != 1) {
    memset(board, 0, sizeof(board));
    for (int i = 0; i < 26; i++) {
        regions[i] = 0;
    }
    if (flag == 2) {
        printf("Input board isn't square\n");
        printf("Enter absolute file path, or q to quit > ");
        scanf("%s", in_str);
        if (strcmp(in_str, "q") == 0) return 0;
        counter = 0;
        for (int i = 0; i < strlen(in_str); i++) {
            if (in_str[i] == '\\') {
                temp[counter] = '\\';
                counter++;
                temp[counter] = '\\';
            } else {
                temp[counter] = in_str[i];
            }
            counter++;
        }
        temp[counter] = '\\0';
        inptr = fopen(temp, "r");
    }

    if (flag == 3) {
        printf("Input board contains invalid characters\n");
        printf("Enter absolute file path, or q to quit > ");
        scanf("%s", in_str);
        if (strcmp(in_str, "q") == 0) return 0;
        counter = 0;
        for (int i = 0; i < strlen(in_str); i++) {
            if (in_str[i] == '\\') {
                temp[counter] = '\\';
                counter++;
                temp[counter] = '\\';
            } else {
                temp[counter] = in_str[i];
            }
            counter++;
        }
        temp[counter] = '\\0';
        inptr = fopen(temp, "r");
    }

    if (flag == 4) {
        printf("Input board doesn't contains the same number of regions as
rows or cols\n");

        printf("Enter absolute file path, or q to quit > ");
        scanf("%s", in_str);
        if (strcmp(in_str, "q") == 0) return 0;
        counter = 0;
        for (int i = 0; i < strlen(in_str); i++) {
            if (in_str[i] == '\\') {
                temp[counter] = '\\';
                counter++;
                temp[counter] = '\\';
            } else {
                temp[counter] = in_str[i];
            }
            counter++;
        }
        temp[counter] = '\\0';
        inptr = fopen(temp, "r");
    }

    while (inptr == NULL) {

```

```

        printf("file read failed\n");
        printf("Enter absolute file path, or q to quit > ");
        scanf("%s", in_str);
        if (strcmp(in_str, "q") == 0) return 0;
        counter = 0;
        for (int i = 0; i < strlen(in_str); i++) {
            if (in_str[i] == '\\') {
                temp[counter] = '\\';
                counter++;
                temp[counter] = '\\';
            } else {
                temp[counter] = in_str[i];
            }
            counter++;
        }
        temp[counter] = '\\0';
        inptr = fopen(temp, "r");
    }

    flag = 1;
    int cols = 0;
    fgets(iter_str, MAX_BOARD_SIZE, inptr);
    bsz = strlen(iter_str);
    if (iter_str[bsz - 1] == '\\n') {
        iter_str[bsz - 1] = '\\0';
        bsz--;
    }
    for (int i = 0; i < bsz; i++) {
        if (iter_str[i] > 90 || iter_str[i] < 65) {
            flag = 3;
            break;
        } else {
            regions[iter_str[i] - 65]++;
        }
    }
    strncat(board, iter_str, bsz);
    cols++;
    while (fgets(iter_str, MAX_BOARD_SIZE, inptr) && flag != 3) {
        int temp_len = strlen(iter_str);
        if (iter_str[temp_len - 1] == '\\n') {
            iter_str[temp_len - 1] = '\\0';
            temp_len--;
        }
        if (bsz != temp_len) {
            flag == 2;
            break;
        }
        for (int i = 0; i < bsz; i++) {
            if (iter_str[i] > 90 || iter_str[i] < 65) {
                flag = 3;
                break;
            } else {
                regions[iter_str[i] - 65]++;
            }
        }
        if (flag == 3) break;
        strncat(board, iter_str, bsz);
        cols++;
    }
    if (cols != bsz) flag = 2;
    int num_regions = 0;
    for (int i = 0; i < 26; i++) {
        if (regions[i] > 0) num_regions++;
    }
    if (num_regions != bsz) flag = 4;
    fclose(inptr);
}
for (int i = 0; i < 26; i++) {
    regions[i] = 0;
}

```

```

    }

    system("cls");
    printf("Regions:\n");
    for (int i = 0; i < bsz; i++) {
        for (int j = 0; j < bsz; j++) {
            printf("%c", board[i * bsz + j]);
        }
        printf("\n");
    }
    printf("\n\033[%dE", bsz);

    create_queens(&q, bsz);
    int cases = 1;

    auto t1 = std::chrono::steady_clock::now();
    char state = 0;
    while (q.buffer[0] < q.num_queens * (q.num_queens - 1)) {
        cases++;
        printf("\033[%dF", q.num_queens);
        move_queen(&q, q.num_queens - 1);
        if (is_valid_board(&q, board, regions) == 1) {
            printf("\033[32m");
        } else {
            printf("\033[39m");
        }
        print_board(&q, board);
        if (is_valid_board(&q, board, regions) == 1) {
            state = 1;
            break;
        }
    }
    auto t2 = std::chrono::steady_clock::now();
    auto elapsed_ms = std::chrono::duration_cast<std::chrono::milliseconds>(t2 - t1);
    printf("\033[0m\nWaktu: %d ms\n", elapsed_ms.count());
    printf("Kasus: %d\n", cases);
    if (state == 0) {
        printf("Tidak ada posisi valid\n");
        return 0;
    };
    printf("Apakah kamu ingin menyimpan hasil ini?(Y/N) > ");
    while ((dump = getchar()) != '\n' && dump != EOF);
    char yn;
    scanf("%c", &yn);
    while (!(yn == 'y' || yn == 'Y' || yn == 'n' || yn == 'N')) {
        printf("Invalid Input\n");
        printf("Apakah kamu ingin menyimpan hasil ini?(Y/N) > ");
        while ((dump = getchar()) != '\n' && dump != EOF);
        scanf("%c", &yn);
    }
    if (yn == 'y' || yn == 'Y') {
        FILE *outptr;
        outptr = fopen("output", "w");
        counter = 0;
        if (outptr != NULL) {
            for (int i = 0; i < bsz; i++) {
                for (int j = 0; j < bsz; j++) {
                    if (q.buffer[counter] == i * bsz + j) {
                        fprintf(outptr, "#");
                        counter++;
                    } else {
                        fprintf(outptr, "%c", board[i * bsz + j]);
                    }
                }
                fprintf(outptr, "\n");
            }
            fclose(outptr);
            printf("Output file written at \\bin");
        } else {

```

```

        printf("output failed\n");
    }
}

dealloc_queens(&q);
return 0;
}

```

queens.h

```

#ifndef QUEENS_H
#define QUEENS_H

#define MAX_BOARD_SIZE 100

typedef struct {
    int num_queens;
    int *buffer;
    int *queens_per_row;
    int *queens_per_col;
} queens;

void create_queens(queens *q, int n);

void dealloc_queens(queens *q);

void move_queen(queens *q, int idx);

char is_valid_board(queens *q, char *board, int *regions);

void print_board(queens *q, char *board);

#endif

```

queens.c

```

#include "queens.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void create_queens(queens *q, int n) {
    q->buffer = (int *)malloc(n * n * sizeof(int));
    if (q->buffer != NULL) {
        q->queens_per_row = (int *)malloc(n * sizeof(int));
        if (q->queens_per_row == NULL) {
            dealloc_queens(q);
            printf("r");
            return;
        }
        q->queens_per_col = (int *)malloc(n * sizeof(int));
        if (q->queens_per_col == NULL) {
            dealloc_queens(q);
            printf("c");
            return;
        }
        q->num_queens = n;
        for (int i = 0; i < q->num_queens; i++) {
            q->buffer[i] = i;
            q->queens_per_col[i] = 1;
            if (i == 0) {
                q->queens_per_row[i] = q->num_queens;
            } else {
                q->queens_per_row[i] = 0;
            }
        }
    }
}

```

```

    } else {
        q->num_queens = 0;
        printf("b");
    }
}

void dealloc_queens(queens *q) {
    if (q->buffer != NULL) {
        free(q->buffer);
        q->buffer = NULL;
    }
    if (q->queens_per_row != NULL) {
        free(q->queens_per_row);
        q->queens_per_row = NULL;
    }
    if (q->queens_per_col != NULL) {
        free(q->queens_per_col);
        q->queens_per_col = NULL;
    }
    q->num_queens = 0;
}

void move_queen(queens *q, int idx) {
    if (idx < 0 || idx >= q->num_queens) return;
    q->queens_per_row[q->buffer[idx] / q->num_queens]--;
    q->queens_per_col[q->buffer[idx] % q->num_queens]--;
    q->buffer[idx]++;
    if (q->buffer[idx] < q->num_queens * q->num_queens) {
        q->queens_per_row[q->buffer[idx] / q->num_queens]++;
        q->queens_per_col[q->buffer[idx] % q->num_queens]++;
    }
    for (int j = idx; j >= 0; j--) {
        if (q->buffer[j] >= q->num_queens * q->num_queens - (q->num_queens - j - 1)) {
            if (j > 0) {
                if (q->buffer[j - 1] < q->num_queens * q->num_queens) {
                    q->queens_per_row[q->buffer[j - 1] / q->num_queens]--;
                    q->queens_per_col[q->buffer[j - 1] % q->num_queens]--;
                }
                q->buffer[j - 1]++;
                if (q->buffer[j - 1] < q->num_queens * q->num_queens) {
                    q->queens_per_row[q->buffer[j - 1] / q->num_queens]++;
                    q->queens_per_col[q->buffer[j - 1] % q->num_queens]++;
                }
            }
            for (int k = j; k < q->num_queens; k++) {
                if (q->buffer[k] < q->num_queens * q->num_queens) {
                    q->queens_per_row[q->buffer[k] / q->num_queens]--;
                    q->queens_per_col[q->buffer[k] % q->num_queens]--;
                }
                q->buffer[k] = q->buffer[j - 1] + 1 + k - j;
                if (q->buffer[k] < q->num_queens * q->num_queens) {
                    q->queens_per_row[q->buffer[k] / q->num_queens]++;
                    q->queens_per_col[q->buffer[k] % q->num_queens]++;
                }
            }
        }
    }
} else {
    break;
}
}
}

```

```

char is_valid_board(queens *q, char *board, int *regions) {
    for (int i = 0; i < 26; i++) {
        regions[i] = 0;
    }

    //check rook-style movement
    if (q->buffer == NULL || q->queens_per_row == NULL || q->queens_per_col == NULL)
return 0;
    for (int i = 0; i < q->num_queens; i++) {
        if (q->queens_per_row[i] != 1 || q->queens_per_col[i] != 1) return 0;
    }

    //check king-style movement
    for (int i = 0; i < q->num_queens; i++) {
        regions[(int)board[q->buffer[i]] - 65]++;
        for (int j = -1; j <= 1; j++) {
            for (int k = -1; k <= 1; k++) {
                if (q->buffer[i] + q->num_queens * j + k > 0 && q->buffer[i]
+ q->num_queens * j + k < q->num_queens * q->num_queens && !(j == 0 && k == 0)) {
                    for (int l = 0; l < q->num_queens; l++) {
                        if (q->buffer[l] == (q->buffer[i] / q-
>num_queens + j) * q->num_queens + q->buffer[i] % q->num_queens + k) return 0;
                    }
                }
            }
        }
    }

    //check regions
    int num_unary_regions = 0;
    for (int i = 0; i < 26; i++) {
        if (regions[i] > 1) return 0;
        if (regions[i] == 1) num_unary_regions++;
    }
    if (num_unary_regions != q->num_queens) return 0;

    return 1;
}

void print_board(queens *q, char *board) {
    int cur_queen_idx = 0;
    for (int row = 0; row < q->num_queens; row++) {
        for (int col = 0; col < q->num_queens; col++) {
            if (q->buffer[cur_queen_idx] / q->num_queens == row && q-
>buffer[cur_queen_idx] % q->num_queens == col) {
                printf("#");
                cur_queen_idx++;
            } else {
                printf("%c", board[row * q->num_queens + col]);
            }
        }
        printf(" %d\n", q->queens_per_row[row]);
    }
    for (int col = 0; col < q->num_queens; col++) {
        printf("%d", q->queens_per_col[col]);
    }
}

```

3. Tangkapan Layar

```
Regions:
ABC
CBA
BAC

ABC 0
CBA 0
### 3
111
Waktu: 38 ms
Kasus: 84
Tidak ada posisi valid
```

```
Regions:
AAAAA
BBBBB
CCCCC
DDDDD
EEEEE

#AAAA 1
BB#BB 1
CCCC# 1
D#DDD 1
EEE#E 1
11111
Waktu: 9946 ms
Kasus: 8097
Apakah kamu ingin menyimpan hasil ini?(Y/N) > |
```


Regions:

AABD

ABDC

ADDD

BCDA

AA#D 1

#BDC 1

ADD# 1

B#DA 1

1111

Waktu: 967 ms

Kasus: 932

Regions:

AA

BB

AA 0

2

11

Waktu: 4 ms

Kasus: 6

Tidak ada posisi valid

```

Regions:
AABDE
DEBAA
DECAA
ACDED
BBCEC

AA#DE 1
#EBAA 1
DECA# 1
A#DED 1
BBC#C 1
11111
Waktu: 63871 ms
Kasus: 22771
Apakah kamu ingin menyimpan hasil ini?(Y/N) >

```

4. Pranala
https://github.com/kalkabena/Tucil1_13524119.git
5. Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.


Nathanael Shane Bennet

Poin	Ya	Tidak
Program berhasil di kompilasi tanpa kesalahan	✓	
Program berhasil di jalankan	✓	
Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
Program memiliki Graphical User Interface (GUI)		✓

Program dapat menyimpan solusi dalam bentuk file gambar		✓
---	--	---